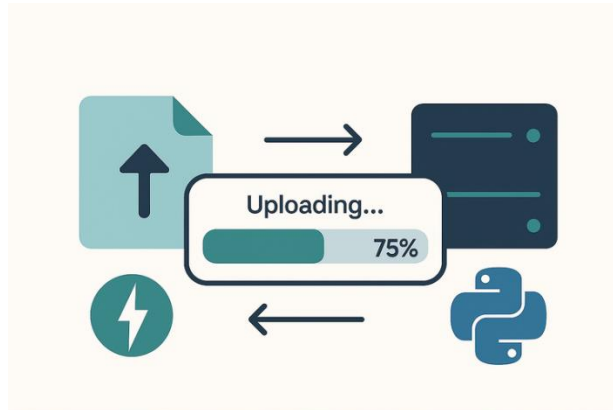


Instituto Tecnológico de Estudios Superiores de Zamora



Practica 2: Política de Seguridad Billey

**Arquitectura de Servicios
Roberto Suárez Sinzun**

**Diego Armando Cazarez Ramírez
21010268**

Dom 31 de mayo de 2026

Índice

- 1.- Descripción General 3
 - 1.1 Roles definidos 3
 - 1.2 Tecnologías de seguridad 3
- 2. Política de Seguridad para BD billety_db 3
- 3. Política de Seguridad de la API REST Billety 5
 - 3.1 Usuarios 5
 - 3.2 Categorías 5
 - 3.3 Movimientos 6
 - 3.4 Planeación Financiera Mensual 6
- 4. Proceso de Autenticación 7
 - 4.1 Mecanismo 7
 - 4.2 Flujo de autenticación 7
- 5. Proceso de Autorización 7
- 6. Configuración del Rate Limiting 8

1.- Descripción General

Billeyty es una API REST de finanzas personales desarrollada con FastAPI y MySQL. La presente política de seguridad define los roles de acceso al sistema, los privilegios asignados a nivel de base de datos y de API REST, así como los mecanismos de autenticación y autorización implementados.

1.1 Roles definidos

El sistema define dos roles con diferentes niveles de acceso:

- **admin** — Acceso total al sistema. Puede gestionar cuentas de cualquier usuario y modificar roles.
- **usuario** — Acceso restringido a sus propios datos. No puede gestionar cuentas ajenas ni modificar roles.

1.2 Tecnologías de seguridad

Componente	Tecnología / Mecanismo
Autenticación	HTTP Basic Auth (correo + contraseña en cada petición)
Hash contraseñas	SHA-256 — nunca se almacena la contraseña en texto plano
Autorización	RoleChecker — verificación de rol en cada endpoint protegido
Conexión a BD	La conexión se establece con las credenciales del usuario MySQL correspondiente al rol autenticado
Rate Limiting	slowapi — límite de solicitudes por dirección IP

2. Política de Seguridad para BD billeyty_db

Se crean dos usuarios de MySQL, uno por cada rol del sistema. Cada usuario tiene privilegios asignados a nivel de tabla siguiendo el principio de mínimo privilegio. La conexión a la base de datos se realiza con las credenciales del usuario MySQL correspondiente al rol del usuario autenticado en la API.

Rol: admin				
Tabla	SELECT	INSERT	UPDATE	DELETE
usuarios	✓	✓	✓	✓
categorias	✓	✓	✓	✓
movimientos	✓	✓	✓	✓
planeacion_mensual	✓	✓	✓	✓

Rol: usuario				
Tabla	SELECT	INSERT	UPDATE	DELETE
usuarios	✓	✓	✓	✗
categorias	✓	✓	✓	✗
movimientos	✓	✓	✓	✗
planeacion_mensual	✓	✓	✓	✗

Nota: El rol usuario no tiene privilegio DELETE en ninguna tabla ya que el sistema no implementa eliminación física. Las bajas se realizan mediante cambios de estatus (estatus = false en usuarios y categorías, estatus = 'cancelado' en movimientos), preservando la integridad del historial financiero.

Restricciones adicionales a nivel de BD:

- movimientos.monto > 0
- movimientos.tipo_movimiento IN ('ingreso', 'gasto')
- movimientos.tipo_pago IN ('efectivo', 'tarjeta_debito', 'tarjeta_credito')
- movimientos.estatus IN ('activo', 'cancelado')
- usuarios.correo UNIQUE
- usuarios.rol IN ('admin', 'usuario')
- planeacion_mensual.mes BETWEEN 1 AND 12
- planeacion_mensual.anio > 0
- UNIQUE (id_usuario, mes, anio) en planeacion_mensual — evita duplicados por período.

3. Política de Seguridad de la API REST Billety

La siguiente matriz define los endpoints disponibles para cada rol. Los endpoints marcados con **X** para el rol usuario retornan HTTP 403 Forbidden. Para los endpoints marcados con **✓** en ambos roles, el sistema valida adicionalmente que el usuario solo pueda acceder a sus propios recursos.

3.1 Usuarios

#	Operación	Endpoint	HTTP	Admin	Usuario
1	Registrar usuario	/usuarios/registrar	POST	✓	✓
2	Login	/usuarios/login	POST	✓	✓
3	Consultar usuario por ID	/usuarios/{id}	GET	✓	✓
4	Desactivar cuenta	/usuarios/{id}/desactivar	PATCH	✓	✓
5	Activar cuenta	/usuarios/{id}/activar	PATCH	✓	X
6	Cambiar rol	/usuarios/{id}/rol	PATCH	✓	X

3.2 Categorías

#	Operación	Endpoint	HTTP	Admin	Usuario
1	Crear categoría	/categorias/crear	POST	✓	✓
2	Editar categoría	/categorias/{id}/editar	PUT	✓	✓
3	Desactivar categoría	/categorias/{id}/desactivar	PATCH	✓	✓
4	Reactivar categoría	/categorias/{id}/reactivar	PATCH	✓	✓
5	Consultar por usuario	/categorias/{id_usuario}	GET	✓	✓
6	Consultar por estado	/categorias/{id_usuario}/estado/{estado}	GET	✓	✓

3.3 Movimientos

#	Operación	Endpoint	HTTP	Admin	Usuario
1	Registrar movimiento	/movimientos/registrar	POST	✓	✓
2	Editar movimiento	/movimientos/editar/{id}	PUT	✓	✓
3	Cancelar movimiento	/movimientos/cancelar/{id}	PATCH	✓	✓
4	Consultar por ID	/movimientos/{id}	GET	✓	✓
5	Consultar por tipo	/movimientos/{id_usuario}/tipo/{tipo}	GET	✓	✓
6	Consultar por mes y año	/movimientos/{id_usuario}/mes/{mes}/anio/{anio}	GET	✓	✓
7	Consultar por categoría	/movimientos/{id_usuario}/categoria/{id_categoria}	GET	✓	✓
8	Consultar por tipo de pago	/movimientos/{id_usuario}/pago/{tipo_pago}	GET	✓	✓

3.4 Planeación Financiera Mensual

#	Operación	Endpoint	HTTP	Admin	Usuario
1	Crear planeación	/planeacion/crear	POST	✓	✓
2	Consultar por ID	/planeacion/{id}	GET	✓	✓
3	Historial de planeaciones	/planeacion/{id_usuario}/historial	GET	✓	✓
4	Consultar por mes y año	/planeacion/{id_usuario}/mes/{mes}/anio/{anio}	GET	✓	✓
5	Editar planeación	/planeacion/{id}/editar	PUT	✓	✓

4. Proceso de Autenticación

4.1 Mecanismo

La API implementa HTTP Basic Authentication. En cada petición a un endpoint protegido, el cliente envía sus credenciales codificadas en Base64 en el encabezado Authorization:

Authorization: Basic base64(correo:password)

El sistema verifica que el correo exista, que el hash SHA-256 de la contraseña coincida con el almacenado, y que el estatus del usuario sea activo. Una vez autenticado, la API establece la conexión a MySQL con el usuario correspondiente al rol (billety_admin o billety_usuario).

4.2 Flujo de autenticación

Paso	Descripción
1	Cliente envía petición con credenciales en el header Authorization (HTTP Basic).
2	getUser extrae las credenciales y consulta la tabla usuarios en la BD.
3	Si el correo no existe, retorna HTTP 401 Unauthorized.
4	Se compara el hash SHA-256 de la contraseña recibida con el almacenado. Si no coincide, retorna HTTP 401.
5	Se verifica que estatus = true. Si la cuenta está inactiva, retorna HTTP 403.
6	Si todo es válido, se retorna el objeto Usuario con su rol para el proceso de autorización.
7	La API conecta a MySQL con billety_admin o billety_usuario según el rol del usuario autenticado.

5. Proceso de Autorización

La autorización se implementa mediante la clase RoleChecker, usada como dependencia en FastAPI. Recibe la lista de roles permitidos para cada endpoint y verifica el rol del usuario autenticado. Si el usuario no tiene el rol requerido, retorna HTTP 403 Forbidden.

Grupo de endpoints	Roles permitidos
POST /usuarios/regar y POST /usuarios/login	Públicos — sin autenticación requerida
PATCH /usuarios/{id}/desactivar	admin, usuario (usuario solo puede desactivar su propia cuenta)
PATCH /usuarios/{id}/activar	admin únicamente
PATCH /usuarios/{id}/rol	admin únicamente
Todos los demás endpoints	admin, usuario (usuario solo accede a sus propios datos)

6. Configuración del Rate Limiting

Se implementa rate limiting por dirección IP utilizando la librería slowapi. Cuando se supera el límite, la API retorna HTTP 429 Too Many Requests.

Endpoint	Límite	Justificación
POST /usuarios/regar	5 / minuto	Prevenir creación masiva de cuentas
POST /usuarios/login	10 / minuto	Proteger contra fuerza bruta
Endpoints GET	60 / minuto	Uso normal de la aplicación
Endpoints POST/PUT/PATCH	30 / minuto	Operaciones de escritura moderadas